

课程笔记

CS336 Lecture 6: Kernels, Triton

基于 Tatsu Hashimoto 授课内容整理

2025 年 5 月



视频作者/频道: Stanford Online

发布日期: 2025-05

视频时长: 1:20:18

视频链接: <https://www.youtube.com/watch?v=E8Mju53VB00>

目录

1 引言：从理论到实践	2
1.1 本章小结	2
2 GPU 架构快速回顾	2
2.1 硬件结构	2
2.2 执行模型	3
2.3 Thread Block 的意义	3
2.4 Wave Quantization 与 SM 利用率	3
2.5 算术强度	4
2.6 本章小结	4
3 基准测试 (Benchmarking)	4
3.1 为什么要基准测试	4
3.2 正确的基准测试方法	4
3.3 基准测试案例：矩阵乘法	5
3.4 基准测试案例：MLP 缩放	5
3.5 本章小结	6
4 性能分析 (Profiling)	6
4.1 从端到端到深入内部	6
4.2 PyTorch 内置 Profiler	6
4.3 Profiling 案例：简单操作	7
4.3.1 矩阵加法	7
4.3.2 矩阵乘法	7
4.3.3 复合操作：cdist	7
4.3.4 GeLU 与 Softmax	7
4.4 Profiling 案例：MLP (Nsight 火焰图)	8
4.5 本章小结	8
5 内核融合 (Kernel Fusion) 动机	8
5.1 仓库-工厂类比	8
5.2 GeLU：融合效果的完美案例	9
5.3 GeLU 性能对比	9
5.4 本章小结	10
6 编写 CUDA 内核	10
6.1 CUDA 编程基础	10
6.2 GeLU 的 CUDA 实现	10
6.3 Python 端启动 CUDA 内核	11
6.4 CUDA GeLU 性能	11
6.5 本章小结	12

7 Triton 内核编写	12
7.1 Triton 简介	12
7.2 GeLU 的 Triton 实现	13
7.2.1 包装器函数 (CPU 端)	13
7.2.2 内核函数 (GPU 端)	13
7.3 PTX: 查看 GPU “汇编” 代码	14
7.4 GeLU 五种实现的完整对比	14
7.5 本章小结	15
8 PyTorch 编译优化 (torch.compile)	15
8.1 自动融合的魔力	15
8.2 torch.compile 的适用场景	16
8.3 本章小结	16
9 Triton Softmax: 行归约操作	16
9.1 从逐元素到行归约	16
9.2 朴素 Softmax 实现与内存分析	16
9.3 Triton Softmax 实现	17
9.3.1 包装器函数	17
9.3.2 内核函数	17
9.4 Softmax 的读写次数详细分析	18
9.5 Softmax 完整性能对比	19
9.6 本章小结	19
10 矩阵乘法补充: Tiling 与 L2 Cache 优化	19
10.1 为什么手写矩阵乘法内核?	19
10.2 Tiling 策略	19
10.3 L2 Cache 友好的遍历顺序	20
10.4 本章小结	20
11 工程实践建议	20
11.1 开发高性能代码的工作流	20
11.2 CPU-GPU 异步执行的实践影响	21
11.3 矩阵维度选择	21
11.4 选择正确的数值精度	21
11.5 本章小结	21
12 总结与延伸	22
12.1 五种实现方式的权衡	22
12.2 全课知识图谱	22
12.3 关键 Takeaways	22
12.4 拓展阅读	22

1 引言：从理论到实践

上一节课 (Lecture 5) 从宏观层面讲解了 GPU 的硬件架构和性能优化原理。本节课将从**实操**角度出发，学习如何编写、度量和优化 GPU 上的高性能代码。核心目标有三个：

1. **掌握基准测试 (Benchmarking) 与性能分析 (Profiling)** ——理解代码在 GPU 上实际的执行方式
2. **理解内核融合 (Kernel Fusion) 的重要性**——用 GeLU 和 Softmax 作为案例
3. **掌握五种实现方式的权衡**——手写 PyTorch、原生 PyTorch、CUDA C++、Triton、torch.compile

本节课的核心信息

编程模型 (PyTorch、Triton、PTX) 与硬件之间存在巨大的抽象鸿沟，这会导致许多“性能之谜”。基准测试帮助我们理解扩展行为，性能分析帮助我们理解 PyTorch 函数的内部机制 (最终归结为 CUDA 内核调用)，查看 PTX 汇编帮助我们理解 CUDA 内核的内部工作。

课程提供了五种实现同一函数的方法：

- **Manual**: 用基本 PyTorch 操作手写 (最慢，因为没有融合)
- **PyTorch native**: 调用 PyTorch 内置函数 (已优化)
- **CUDA C++**: 直接用 CUDA 编写内核
- **Triton**: 用 Python 编写 GPU 内核
- **torch.compile**: 让编译器自动优化手写代码

与上一节课的关系

Lecture 5 建立了 GPU 硬件的心智模型 (SM、内存层次、Tiling、Fusion 等)。本节课用代码将这些概念“落地”——你将亲手编写内核，用分析工具验证理论预测，并直接观察不同实现的性能差异。

1.1 本章小结

本课以 MLP (线性层 + GeLU 激活堆叠) 作为贯穿全课的实验对象，GeLU (逐元素操作) 和 Softmax (行归约操作) 作为内核编写的核心案例。掌握基准测试和性能分析是编写高性能代码的**第一步，也是最重要的一步**。

2 GPU 架构快速回顾

在进入本节课的核心内容之前，先快速回顾上一节课的关键概念。

2.1 硬件结构

GPU 的核心结构包括：

- **SM (Streaming Multiprocessor)**: GPU 的计算单元，A100 有 108 个 SM

- **内存层次:**

- DRAM (全局内存): A100 有 80GB, 大但慢
- L2 Cache: A100 有 40MB
- L1 Cache / Shared Memory: A100 每个 SM 有 192KB, 小但快

2.2 执行模型

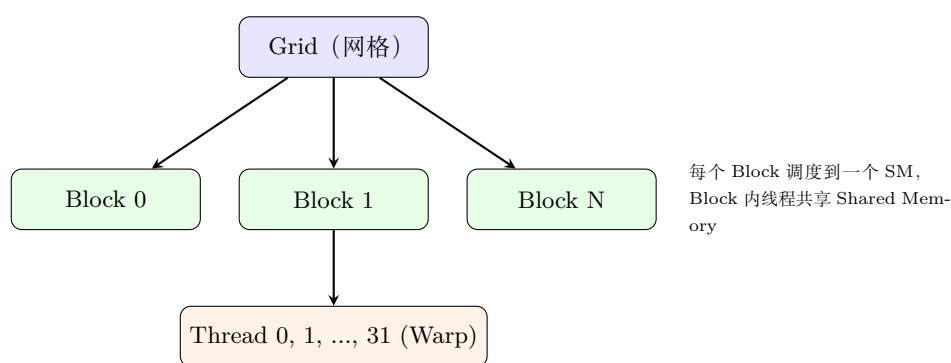


图 1: GPU 执行模型层次: Grid $\rightarrow$ Block $\rightarrow$ Thread

- **Thread (线程):** 执行 $f(i)$ 的最小单位
- **Thread Block (又称 CTA):** 调度到单个 SM 上的线程集合, Block 内线程共享 Shared Memory
- **Grid:** 所有 Thread Block 的集合

2.3 Thread Block 的意义

为什么需要 Thread Block 而不是直接使用裸线程?

1. Block 内线程拥有**共享内存** (Shared Memory), 速度与 L1 Cache 相当
2. Block 内线程可以**同步** (synchronize), 但跨 Block 无法同步
3. 需要线程间通信的操作 (如矩阵乘法) 应安排在同一个 Block 内

2.4 Wave Quantization 与 SM 利用率

Thread Block 以“波 (wave)”为单位调度到 SM 上。理想情况下, 每一波应填满所有 SM。

Wave Quantization 陷阱

如果 Block 数不能被 SM 数量整除, 最后一波会有 SM 空闲。经验法则: Block 数应 $\geq 4 \times$ SM 数量, 以确保高利用率。

2.5 算术强度

算术强度

- 高算术强度 $\Rightarrow$ **计算受限** (compute-bound) ——这是好事
- 低算术强度 $\Rightarrow$ **内存受限** (memory-bound) ——GPU 计算单元闲置

一般规则：矩阵乘法是 compute-bound，其余几乎所有操作都是 memory-bound。

2.6 本章小结

GPU 执行模型的核心是：写一个函数 $f(i)$ ，GPU 对所有 $i = 0, \dots, N - 1$ 并行执行。Block 提供了快速的共享内存和同步机制，但跨 Block 通信代价高昂。性能优化的关键在于最小化全局内存读写、最大化算术强度。

3 基准测试 (Benchmarking)

3.1 为什么要基准测试

基准测试测量操作的**端到端墙钟时间** (wall-clock time)。虽然简单，但它是性能优化的基础：

- 比较不同实现的速度 (“CUDA 版本比 PyTorch 快吗? ”)
- 理解性能如何随参数缩放 (“矩阵增大一倍，时间增加多少? ”)

不能只靠理论推断性能

你可以阅读 spec sheet (营销材料) 和论文，但实际性能取决于你的**库版本、硬件和工作负载**。没有什么能替代实际的基准测试。

3.2 正确的基准测试方法

一个正确的 GPU 基准测试函数需要注意以下几点：

```
1 def benchmark(description, run, num_warmups=1, num_trials=3):
2     # 1. Warmup: 避免测量启动开销
3     for _ in range(num_warmups):
4         run()
5     if torch.cuda.is_available():
6         torch.cuda.synchronize() # 等待 GPU 完成
7
8     # 2. 计时
9     times = []
10    for trial in range(num_trials):
11        start_time = time.time()
12        run()
13        if torch.cuda.is_available():
14            torch.cuda.synchronize() # 重要!
15        end_time = time.time()
16        times.append((end_time - start_time) * 1000)
```

```

17
18     return mean(times)

```

Listing 1: 基准测试函数模板 (Python)

基准测试两大要点

1. **Warmup**: 首次运行时, PyTorch 会在后台编译 CUDA 代码、发送指令到 GPU 等。这些一次性开销不应计入稳态性能。
2. **torch.cuda.synchronize()**: GPU 和 CPU 是两个独立的计算设备。CPU 发送 CUDA 内核到 GPU 后会立即返回 (异步执行)。如果不调用 `synchronize()`, 你测量的是 CPU 端入队的时间, 而非 GPU 端执行的时间。

3.3 基准测试案例: 矩阵乘法

对方阵矩阵乘法进行基准测试, 可以观察到性能随维度的变化:

矩阵维度	典型耗时趋势
1024	基线
2048	~3-4x 增长
4096	~8-12x 增长
8192	显著增长, 接近理论 $O(N^3)$
16384	非常大, 充分利用 GPU

表 1: 矩阵乘法的性能随维度缩放

3.4 基准测试案例: MLP 缩放

用一个简单的 MLP (Linear → GeLU → Linear → GeLU → ...) 作为实验对象, 分别缩放不同参数:

```

1 class MLP(nn.Module):
2     def __init__(self, dim, num_layers):
3         super().__init__()
4         self.layers = nn.ModuleList(
5             [nn.Linear(dim, dim) for _ in range(num_layers)]
6         )
7
8     def forward(self, x):
9         for layer in self.layers:
10             x = layer(x)
11             x = torch.nn.functional.gelu(x)
12         return x

```

Listing 2: MLP 定义

缩放实验的发现:

- **步数 (num_steps)**: 线性缩放—— $2\times$ 步数 $\approx 2\times$ 时间
- **层数 (num_layers)**: 线性缩放——每层增加固定时间
- **批大小 (batch_size)**: 小批量时增长缓慢 (GPU 未充分利用), 大批量时接近线性
- **维度 (dim)**: 增长不太可预测——涉及矩阵乘法的非均匀 CUDA 内核调度

性能不总是可预测的

由于 CUDA 内核调度、硬件特性、缓存行为等因素, 基准测试结果不总是符合简单的理论预测。这正是实际基准测试不可替代的原因。

3.5 本章小结

基准测试是性能优化的第一步。正确的基准测试需要 warmup 和 CUDA synchronize。它告诉你“多快”, 但不告诉你“为什么快/慢”——这需要 Profiling。

4 性能分析 (Profiling)

4.1 从端到端到深入内部

基准测试给出端到端时间, 性能分析则揭示**时间花在了哪里**。更深层地, 性能分析让你看到 PyTorch 高层调用背后实际执行的 CUDA 内核。

Profiling 的两层价值

1. **显式价值**: 哪些函数占用了最多时间? 应该优化哪里?
2. **隐式价值**: PyTorch 调用在底层映射为哪些 CUDA 内核? 理解这些映射关系帮助你建立对 GPU 执行的直觉。

4.2 PyTorch 内置 Profiler

PyTorch 提供了方便的内置性能分析器, 无需离开 Python 环境:

```

1 def profile(description, run, num_warmups=1, with_stack=False):
2     # Warmup
3     for _ in range(num_warmups):
4         run()
5     torch.cuda.synchronize()
6
7     # Profile
8     with torch.profiler.profile(
9         activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
10        with_stack=with_stack,
11    ) as prof:
12        run()
13        torch.cuda.synchronize()
14
15    # Print results

```



```

16     table = prof.key_averages().table(
17         sort_by="cuda_time_total",
18         row_limit=10
19     )
20     return table

```

Listing 3: PyTorch Profiler 使用模板

4.3 Profiling 案例：简单操作

4.3.1 矩阵加法

Profile 矩阵加法 $a + b$ (维度 2048×2048):

- PyTorch 层面：调用 `aten::add` (C++ 接口)
- GPU 层面：启动一个 CUDA 内核 `vectorized_elementwise_kernel`
- 这是最简单的一对一映射：一个 PyTorch 操作 $\rightarrow$ 一个 CUDA 内核

4.3.2 矩阵乘法

Profile 矩阵乘法 $a @ b$:

- 大矩阵(2048×2048):调用 CUTLASS 库的内核,如 `cutlass_80_simt_sgemm_256x128_8x4_nn_align1`
- 小矩阵 (128×128): 调用不同的内核, 如 `xmma_gemm_f32f32`
- 内核名称本身包含丰富信息: `cutlass` (NVIDIA 线性代数库)、`256x128` (tile 大小)

不同维度调度不同内核

PyTorch 的矩阵乘法 $a @ b$ 在高层只是一行代码,但底层会根据矩阵维度、硬件类型等因素调度到完全不同的 CUDA 内核。`torch.compile` 甚至可以自动微基准测试不同的内核,选择最优的——这就是为什么它能给你“免费”的 10% 加速。

4.3.3 复合操作: `cdist`

`torch.cdist(a, b)` 计算两组向量的欧氏距离矩阵:

- 映射到 `aten::cdist` $\rightarrow$ `aten::euclidean_dist`
- 分解为多个原语: `aten::mm` (矩阵乘法, 占 78% GPU 时间)、`aten::pow` (5%)、`aten::sum` (3%)、数组拼接 (6%)
- 这告诉我们: 如果要优化 `cdist`, 应该集中精力在矩阵乘法上

4.3.4 GeLU 与 Softmax

GeLU ($a + b$ 后接 GeLU) 和 Softmax 是本课的两个核心案例,将在后续章节详细分析。

4.4 Profiling 案例：MLP (Nsight 火焰图)

对一个较大的 MLP (dim = 2048, 64 层, batch = 1024) 进行 Profiling, 使用 NVIDIA Nsight 可视化工具可以看到**火焰图** (flame graph):

CPU 与 GPU 的异步执行

Profiling 揭示了一个关键事实: CPU 和 GPU 是**异步执行的**。CPU 会持续向 GPU 发送内核调用命令 (“run this next, run this next...”), 远远领先于 GPU 的实际执行。例如: 当 GPU 还在执行 Layer 1 时, CPU 可能已经在排队 Layer 9 的命令了。

print 语句的隐藏性能影响

在训练循环中打印损失值 (如 `print(loss.item())`) 看似无害, 但会**强制 CPU-GPU 同步**——CPU 必须等待 GPU 计算完损失值才能打印。这会破坏异步执行, 让 CPU 和 GPU 变成锁步 (lock-step) 执行, 显著降低性能。

正确做法: 只在必要时 (如每 N 步) 打印, 或使用日志缓冲区。

4.5 本章小结

性能分析是定位瓶颈的关键工具。它让你看到 PyTorch 高层调用背后的 CUDA 内核, 理解 CPU-GPU 异步执行模型, 发现隐藏的性能陷阱 (如同步开销)。**每次修改代码后, 都应该重新基准测试和 Profile。**

5 内核融合 (Kernel Fusion) 动机

5.1 仓库-工厂类比

Horace He 的经典博客文章 *Making Deep Learning Go Brrrrr* 用工厂比喻解释了内存瓶颈:

- **仓库 (Warehouse)** = DRAM (全局内存) ——大、慢
- **工厂 (Factory)** = SRAM (片上计算) ——小、快

如果每完成一步加工就把半成品运回仓库, 再从仓库取出来做下一步, 搬运成本远超加工成本。如果我们把多步加工**融合**在工厂内一次完成, 只需要一次搬运。

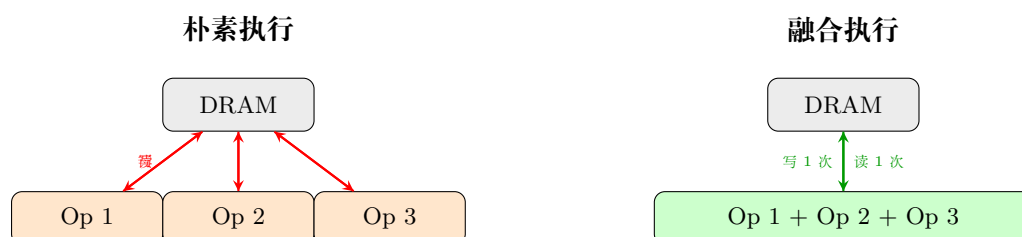


图 2: 内核融合的核心思想: 减少全局内存读写次数

核心原则：组织计算以最小化读写

GPU 性能优化的首要原则是减少全局内存的读写次数。内核融合是实现这一目标的主要手段——将多个操作合并为一个 CUDA 内核，中间结果保留在寄存器或 Shared Memory 中。

5.2 GeLU：融合效果的完美案例

GeLU (Gaussian Error Linear Unit) 是 Transformer 中广泛使用的激活函数。其 tanh 近似版本为：

$$\text{GeLU}(x) = 0.5 \cdot x \cdot \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3) \right) \right)$$

- x ：输入值
- $0.79788456 \approx \sqrt{2/\pi}$ ：常数系数
- 0.044715 ：近似修正系数

手写实现（未融合）：

```
1 def manual_gelu(x):
2     return 0.5 * x * (1 + torch.tanh(
3         0.79788456 * (x + 0.044715 * x * x * x)
4     ))
```

Listing 4: 手写 GeLU（多个独立 CUDA 内核）

PyTorch 内置实现（已融合）：

```
1 def pytorch_gelu(x):
2     return torch.nn.functional.gelu(x, approximate="tanh")
```

Listing 5: PyTorch 内置 GeLU（单个 CUDA 内核）

5.3 GeLU 性能对比

基准测试结果（ 16384×16384 矩阵）：

实现	耗时 (ms)	相对于 PyTorch
Manual GeLU（手写）	~8.1	7.4x 慢
PyTorch GeLU（内置）	~1.1	1.0x（基线）

表 2: GeLU 融合 vs 未融合的性能差异

Profiling 证实了原因：

- **Manual GeLU**：启动多个 CUDA 内核（乘法 $\times 3$ 、加法、tanh），每个内核都要从全局内存读写
- **PyTorch GeLU**：只启动一个 CUDA 内核，所有计算在内核内部完成

8 倍差距仅来自内存读写开销

手写 GeLU 和 PyTorch GeLU 计算的 FLOPs 完全相同，8 倍的性能差距完全来自多余的全局内存读写。这是 memory-bound 操作的典型特征。

5.4 本章小结

内核融合是 GPU 性能优化的核心技术之一。对于 memory-bound 操作（如 GeLU），将多个操作融合为一个 CUDA 内核可以带来数倍的加速。PyTorch 内置函数通常已做了融合，但自定义操作需要手动融合。

6 编写 CUDA 内核

6.1 CUDA 编程基础

CUDA 是 NVIDIA 的 C/C++ API，用于编程 GPU。核心思想很简单：写一个函数 f ，CUDA 对所有元素并行执行 f 。

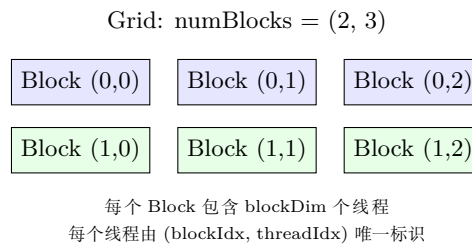


图 3: CUDA Grid-Block-Thread 组织结构

关键概念：

- blockIdx: 当前 Block 在 Grid 中的索引
- blockDim: 每个 Block 的线程数
- threadIdx: 当前线程在 Block 中的索引
- 全局线程索引: $i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$

6.2 GeLU 的 CUDA 实现

```
1 __global__ void gelu_kernel(  
2     const float* in, float* out, int num_elements  
3 ) {  
4     // 计算全局线程索引  
5     int i = blockIdx.x * blockDim.x + threadIdx.x;  
6  
7     // 边界检查  
8     if (i < num_elements) {  
9         float x = in[i];  
10        float a = 0.79788456f * (x + 0.044715f * x * x * x);
```

```

11     float tanh_a = tanhf(a);
12     out[i] = 0.5f * x * (1.0f + tanh_a);
13 }
14 }

```

Listing 6: GeLU CUDA 内核 (gelu.cu)

CUDA 内核的关键模式

几乎所有 CUDA 内核都遵循以下模式：

1. 计算全局索引 i : `blockIdx.x * blockDim.x + threadIdx.x`
2. 边界检查: `if (i < num_elements)`——因为最后一个 Block 可能有多余的线程
3. 执行计算: 从输入指针读取、计算、写入输出指针

`__global__` 关键字标识该函数为 CUDA 内核函数。

6.3 Python 端启动 CUDA 内核

PyTorch 提供了 `load_inline` 工具，可以在 Python 中直接编译和调用 CUDA 代码：

```

1  from torch.utils.cpp_extension import load_inline
2
3  # CUDA 源码 (字符串形式)
4  cuda_src = open("gelu.cu").read()
5  cpp_src = "torch::Tensor gelu(torch::Tensor x);"
6
7  # 编译为 Python 模块
8  module = load_inline(
9      cuda_sources=[cuda_src],
10     cpp_sources=[cpp_src],
11     functions=["gelu"],
12     extra_cflags=["-O2"],
13     name="inline_gelu",
14 )
15
16 cuda_gelu = module.gelu # 现在可以像普通函数一样调用

```

Listing 7: 在 Python 中编译和使用 CUDA 代码

6.4 CUDA GeLU 性能

Profiling CUDA GeLU 的结果显示，我们的自写内核确实只启动了一个 **CUDA 内核**（名为 `gelu_kernel`），所有计算在内核内部完成。与 PyTorch 的差距来自于：

- PyTorch 使用了更高效的数学库函数（如 `tanhf` 的向量化版本）
- PyTorch 的内核做了更细致的**向量化加载**——一次从内存读取多个连续元素
- PyTorch 内核可能使用了**更优的线程配置**和**寄存器分配策略**

实现	耗时 (ms)	说明
Manual (手写 PyTorch)	~8.1	多个内核, 未融合
CUDA (自写内核)	~1.8	单内核, 已融合
PyTorch (内置)	~1.1	单内核, 高度优化

表 3: GeLU 三种实现的性能对比

我们的 CUDA 实现已经比手写 PyTorch 快了 4.5 倍, 但仍不如 PyTorch 内置版本。PyTorch 的实现做了更多底层优化 (如向量化加载、更优的数学库函数)。

逐元素操作在 CUDA 中很简单, 但...

GeLU 是逐元素 (elementwise) 操作——每个输出元素只依赖一个输入元素, 因此 CUDA 实现很直接。但大多数有趣的操作 (矩阵乘法、softmax、RMSNorm) 需要读取多个值, 涉及共享内存管理、线程同步等复杂问题。

6.5 本章小结

CUDA 让你直接控制 GPU 上的每个线程, 实现内核融合。对于逐元素操作, CUDA 编程相对简单, 主要是坐标计算和边界检查。但对于需要线程间通信的操作, CUDA 编程的复杂度会急剧上升——这正是 Triton 出现的原因。

7 Triton 内核编写

7.1 Triton 简介

Triton 由 OpenAI 于 2021 年发布, 目标是让 GPU 编程更易上手:

- 用 Python 编写——无需 C++
- 以 Block 为思考单位——而非线程
- 编译器自动处理许多底层细节

职责	CUDA	Triton
内存合并 (Coalescing)	手动	自动
共享内存管理	手动	自动
SM 内线程调度	手动	自动
跨 SM 调度	手动	手动

表 4: CUDA vs Triton 的职责划分

Triton 的定位

Triton 在 CUDA 和 PyTorch 之间找到了一个甜蜜点：比 PyTorch 更底层（可以实现自定义融合内核），比 CUDA 更高层（编译器处理内存合并和共享内存）。对于许多操作，Triton 的性能可以超越 PyTorch 内置实现。

7.2 GeLU 的 Triton 实现

7.2.1 包装器函数（CPU 端）

```
1 def triton_gelu(x: torch.Tensor):
2     assert x.is_cuda
3     assert x.is_contiguous()
4
5     y = torch.empty_like(x) # 分配输出
6
7     num_elements = x.numel()
8     block_size = 1024      # 每个 Block 处理的元素数
9     num_blocks = triton.cdiv(num_elements, block_size)
10
11     # 启动内核
12     triton_gelu_kernel[(num_blocks,)](
13         x, y, num_elements, BLOCK_SIZE=block_size
14     )
15     return y
```

Listing 8: Triton GeLU 包装器

7.2.2 内核函数（GPU 端）

```
1 @triton.jit
2 def triton_gelu_kernel(x_ptr, y_ptr, num_elements,
3                        BLOCK_SIZE: tl.constexpr):
4     # 计算当前 Block 的起始位置
5     pid = tl.program_id(axis=0)
6     block_start = pid * BLOCK_SIZE
7
8     # 当前 Block 负责的元素索引（向量！）
9     offsets = block_start + tl.arange(0, BLOCK_SIZE)
10
11     # 边界掩码
12     mask = offsets < num_elements
13
14     # 读取数据
15     x = tl.load(x_ptr + offsets, mask=mask)
16
17     # 计算 GeLU（用 exp 实现 tanh）
18     a = 0.79788456 * (x + 0.044715 * x * x * x)
19     exp = tl.exp(2 * a)
```

```

20     tanh = (exp - 1) / (exp + 1)
21     y = 0.5 * x * (1 + tanh)
22
23     # 写回结果
24     tl.store(y_ptr + offsets, y, mask=mask)

```

Listing 9: Triton GeLU 内核

Triton vs CUDA 的关键区别

- **CUDA**: 每个线程处理一个元素，索引 i 是标量
- **Triton**: 每个 Block 处理一组元素，`offsets` 是向量

Triton 让你在 Block 层面思考，编译器自动决定如何将向量操作映射到线程。这使得编译器可以执行更多优化（如 thread coarsening）。

7.3 PTX: 查看 GPU “汇编” 代码

PTX (Parallel Thread Execution) 是 NVIDIA GPU 的虚拟指令集，类似于 CPU 的汇编语言。Triton 编译后生成 PTX 代码，我们可以检查它来理解 GPU 实际执行的操作：

PTX 代码的关键特征

- `ld.global.*`: 从全局内存读取
- `st.global.*`: 写入全局内存
- `%ctaid.x`: Block 索引（对应 `blockIdx.x`）
- `%tid.x`: 线程索引（对应 `threadIdx.x`）
- `%f*`: 浮点寄存器，`%r*`: 整数寄存器

一个重要发现：一个线程同时处理 8 个元素（thread coarsening）——这是 Triton 编译器自动执行的优化，在 CUDA 中需要手动实现。

7.4 GeLU 五种实现的完整对比

实现	耗时 (ms)	语言	融合	说明
Manual	~8.1	Python	否	多个 CUDA 内核
CUDA	~1.8	C++	是	单内核，手动编写
Triton	~1.5	Python	是	单内核，编译器优化
torch.compile	~1.47	Python	是	自动生成 Triton 代码
PyTorch	~1.1	C++	是	高度手工优化

表 5: GeLU 五种实现的完整性能对比（ 16384×16384 矩阵）

Triton 实际上比我们的 CUDA 实现更快

虽然 CUDA 给了你“完全控制”，但 Triton 编译器可以执行 thread coarsening、自动内存合并等优化。对于简单内核，编译器的优化往往比手写的天真 CUDA 代码更好。当然，精心优化的 CUDA 代码可以更快，但开发成本高得多。

7.5 本章小结

Triton 提供了 CUDA 级别的性能和 Python 级别的开发体验。它以 Block 为编程单元，编译器自动处理内存合并和共享内存管理。对于逐元素操作，Triton 代码的结构与 CUDA 非常相似，但用 Python 编写且更易调试。

8 PyTorch 编译优化 (torch.compile)

8.1 自动融合的魔力

`torch.compile` 是 PyTorch 2.0 引入的 JIT 编译器，可以**自动**将朴素 PyTorch 代码优化为融合的 Triton 内核：

```
1 # 原始代码 (未融合, 约 8.1ms)
2 def manual_gelu(x):
3     return 0.5 * x * (1 + torch.tanh(
4         0.79788456 * (x + 0.044715 * x * x * x)
5     ))
6
7 # 一行代码即可优化 (约 1.47ms)
8 compiled_gelu = torch.compile(manual_gelu)
```

Listing 10: torch.compile 使用示例

`torch.compile` 在底层做了什么？

1. 分析 Python 代码的计算图
2. 识别可融合的操作
3. 自动生成优化的 Triton 内核代码
4. 编译并缓存生成的代码

Profiling 证实：编译后的 GeLU 在底层调用了名为 `fused_add_multiply_tanh` 的 Triton 内核——与我们手写的 Triton 代码做了类似的融合，但自动生成的代码略有优化。

8.2 torch.compile 的适用场景

何时用 torch.compile，何时手写 Triton？

- **简单的算子融合**（如 GeLU、pointwise 操作链）：torch.compile 完全够用，不需要手写内核
- **矩阵乘法调度优化**：torch.compile 可以自动选择最优的内核实现，带来约 10% 免费加速
- **复杂的算法优化**（如 FlashAttention）：仍需手写 Triton/CUDA——编译器无法自动发现 online softmax 这样的算法创新
- **硬件特定优化**（如 FlashAttention-3 利用 H100 特性）：需要手写底层代码

不要盲目手写 CUDA 内核

“不要回家后就给语言模型的每个组件都写 CUDA 内核——那可能不是你时间的最佳用途。”但如果你在开发新架构，某个复杂组件的 GPU 利用率很低，而你认为可以改善——那就是拿出 Triton 的时候。

8.3 本章小结

torch.compile 是“低垂的果实”——一行代码就能获得接近手写内核的性能。它在底层生成 Triton 代码来实现算子融合。对于标准操作，优先使用 torch.compile，只在需要非平凡优化时才手写 Triton 内核。

9 Triton Softmax：行归约操作

9.1 从逐元素到行归约

前面的 GeLU 是**逐元素**操作——每个输出只依赖一个输入。Softmax 则是**行归约**操作——每个输出依赖**整行**的所有输入：

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(\mathbf{x})}}{\sum_j e^{x_j - \max(\mathbf{x})}}$$

- x_i ：第 i 个元素
- $\max(\mathbf{x})$ ：整行的最大值（数值稳定性）
- $\sum_j$ ：整行的求和

这意味着需要跨多个元素**通信**（求最大值、求和），不能简单地让每个线程独立工作。

9.2 朴素 Softmax 实现与内存分析

```
1 def manual_softmax(x):
2     M, N = x.shape
3     x_max = x.max(dim=1)[0]          # MN reads, M writes
```

```

4     x = x - x_max[:, None]           # MN+M reads, MN writes
5     numerator = torch.exp(x)         # MN reads, MN writes
6     denominator = numerator.sum(dim=1) # MN reads, M writes
7     y = numerator / denominator[:, None] # MN+M reads, MN writes
8     # Total: 5MN + 2M reads, 3MN + 2M writes
9     return y

```

Listing 11: 朴素 Softmax 实现及内存读写分析

朴素 Softmax 的内存开销

朴素实现需要 $5MN + 2M$ 次读和 $3MN + 2M$ 次写（全局内存访问）。而理论最优只需 MN 次读和 MN 次写——潜在的 4 倍加速！

9.3 Triton Softmax 实现

核心设计思想：**每个 Block 处理一行**。如果行的宽度能放入一个 Block（即放入 SM 的 Shared Memory），那么整行的 max、sum、归一化都可以在 Block 内完成，只需一次全局内存读和一次写。

9.3.1 包装器函数

```

1 def triton_softmax(x):
2     y = torch.empty_like(x)
3     M, N = x.shape
4     block_size = triton.next_power_of_2(N) # 列数向上取整到 2 的幂
5     num_blocks = M                        # 每行一个 Block
6
7     triton_softmax_kernel[(M,)](
8         x_ptr=x, y_ptr=y,
9         x_row_stride=x.stride(0),
10        y_row_stride=y.stride(0),
11        num_cols=N, BLOCK_SIZE=block_size
12    )
13    return y

```

Listing 12: Triton Softmax 包装器

9.3.2 内核函数

```

1 @triton.jit
2 def triton_softmax_kernel(x_ptr, y_ptr,
3     x_row_stride, y_row_stride,
4     num_cols, BLOCK_SIZE: tl.constexpr):
5
6     # 当前处理哪一行
7     row_idx = tl.program_id(0)
8     col_offsets = tl.arange(0, BLOCK_SIZE)
9
10    # 读取整行到 SM 内部

```

```

11     x_start_ptr = x_ptr + row_idx * x_row_stride
12     x_ptrs = x_start_ptr + col_offsets
13     x_row = tl.load(x_ptrs,
14                     mask=col_offsets < num_cols,
15                     other=float("-inf")) # 超出列数的部分用 -inf 填充
16
17     # 在 SM 内部完成所有计算
18     x_row = x_row - tl.max(x_row, axis=0) # 数值稳定
19     numerator = tl.exp(x_row)
20     denominator = tl.sum(numerator, axis=0)
21     y_row = numerator / denominator
22
23     # 写回全局内存
24     y_start_ptr = y_ptr + row_idx * y_row_stride
25     y_ptrs = y_start_ptr + col_offsets
26     tl.store(y_ptrs, y_row, mask=col_offsets < num_cols)

```

Listing 13: Triton Softmax 内核——每个 Block 处理一行

Triton Softmax 的设计精髓

1. 一行 = 一个 Block: 利用 Block 内的 Shared Memory 完成所有归约操作
2. 只需一次全局读写: 整行数据一次性加载到 SM, 计算完成后一次性写回
3. 超出列数的元素用 $-\infty$ 填充: 确保 `tl.max` 和 `tl.exp` 的正确性 ($e^{-\infty} = 0$)

Block Size 为什么是 `next_power_of_2(N)` ?

Triton 要求 Block Size 必须是 2 的幂 (这是硬件要求)。使用 `triton.next_power_of_2(N)` 确保 Block 足够大以容纳整行, 多余的位置用 `mask` 和 `other=float("-inf")` 处理。

9.4 Softmax 的读写次数详细分析

对 $M \times N$ 矩阵的逐行 Softmax, 我们可以精确分析全局内存的读写次数:

操作	读次数	写次数
<code>x.max(dim=1)</code>	MN	M
<code>x - x_max[:, None]</code>	$MN + M$	MN
<code>torch.exp(x)</code>	MN	MN
<code>numerator.sum(dim=1)</code>	MN	M
<code>numerator / denom[:, None]</code>	$MN + M$	MN
合计	$5MN + 2M$	$3MN + 2M$
融合最优	MN	MN
加速比	$\sim 4\times$	

表 6: 朴素 vs 融合 Softmax 的全局内存访问分析

这个分析解释了为什么融合版本的 Softmax 加速比约为 2-3 倍（理论上限约 4 倍）——实际中融合内核还有一些不可避免的开销（如 Block 调度、掩码处理）。

9.5 Softmax 完整性能对比

实现	耗时 (ms)	相对于 PyTorch
Manual (朴素 PyTorch)	~3.7	2.5x 慢
Triton (自写内核)	~1.9	1.3x 慢
PyTorch (内置)	~1.5	1.0x
torch.compile	~1.3	0.87x (更快!)

表 7: Softmax 四种实现的性能对比

值得注意的是：

- `torch.compile` 在 Softmax 上甚至超越了 PyTorch 内置实现——因为它知道输入的 shape，可以做更针对性的优化
- 朴素实现的 3.7ms 到融合实现的 1.3-1.9ms，约 2-3x 加速，不如 GeLU 的 8x——因为 Softmax 本身就有归约操作，理论上限约 4x

本实现假设行宽 $\leq$ Block Size

当矩阵的列数非常大（超过 SM 的 Shared Memory 容量）时，单个 Block 无法容纳整行。此时需要更复杂的多 Block 协作方案，如 FlashAttention 中使用的 online softmax 和 tiling 策略。

9.6 本章小结

Softmax 展示了 Triton 处理行归约操作的能力。核心设计是“一行 = 一个 Block”：整行数据加载到 SM 内部，在高速内存中完成所有归约计算，最后一次性写回。当行宽适中时，这种设计非常高效。

10 矩阵乘法补充：Tiling 与 L2 Cache 优化

10.1 为什么手写矩阵乘法内核？

PyTorch 内置的矩阵乘法（调用 cuBLAS/CUTLASS）已经高度优化。那为什么还要手写？

答案：融合。当你需要计算 $\text{GeLU}(A \cdot B)$ 时，朴素做法是先算 $A \cdot B$ （写回全局内存），再算 GeLU（从全局内存读取）。如果你有自己的矩阵乘法内核，就可以在同一个内核中完成矩阵乘法和 GeLU，避免中间结果写回全局内存。

10.2 Tiling 策略

矩阵乘法的 Tiling 策略（详见 Lecture 5）：

1. 将 A 、 B 矩阵切分为小块 (tile)
2. 将 tile 加载到 Shared Memory
3. 在 Shared Memory 中执行小矩阵乘法
4. 累加部分和
5. 写回结果

10.3 L2 Cache 友好的遍历顺序

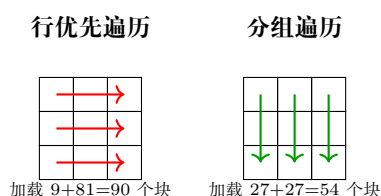


图 4: 不同遍历顺序对 L2 Cache 命中率的影响

分组遍历 (grouped ordering) 让相邻的 tile 共享更多的 A 或 B 矩阵块，从而提高 L2 Cache 命中率。

10.4 本章小结

矩阵乘法是 GPU 上最受优化的操作，但手写内核仍有价值——主要用于与其他操作的融合。Tiling 和 L2 Cache 友好的遍历顺序是两个关键优化策略。

11 工程实践建议

11.1 开发高性能代码的工作流

基于本课的内容，Hashimoto 教授推荐以下工作流：

1. **先用 PyTorch 写正确的代码**：确保逻辑正确，编写测试用例
2. **基准测试**：测量端到端性能，建立 baseline
3. **Profile**：找到真正的瓶颈（不要凭直觉猜测）
4. **尝试 torch.compile**：一行代码可能就够了
5. **必要时写 Triton 内核**：仅对 Profile 确认的瓶颈操作
6. **重新基准测试**：验证优化效果

避免过早优化

“有些同学花了三小时优化一个他们**以为**是瓶颈的组件，结果发现根本不是。” 始终让 Profiling 数据指导你的优化方向。

11.2 CPU-GPU 异步执行的实践影响

CPU-GPU 异步执行模型对日常开发有几个重要影响：

避免不必要的 CPU-GPU 同步

以下操作会强制同步，应谨慎使用：

- `print(tensor.item())` 或 `print(loss)`：需要将 GPU 数据传回 CPU
- `if tensor > threshold:`：需要在 CPU 上评估条件
- `tensor.numpy()`：GPU tensor 到 NumPy 数组的转换
- 任何在 CPU 上访问 GPU tensor 值的操作

11.3 矩阵维度选择

来自上一节课的关键建议，在实践中极为重要：

- 矩阵维度选择 **64/128/256 的倍数**
- 避免质数或不规则的维度
- nanoGPT 案例：词表大小从 50257 $\rightarrow$ 50304（最近的 64 的倍数），获得 25% 加速

11.4 选择正确的数值精度

操作类型	推荐精度	原因
矩阵乘法	FP16/BF16 输入, FP32 累加	Tensor Core 原生支持
Pointwise (ReLU, GeLU 等)	FP16/BF16	减少内存带宽需求
归约 (sum, softmax, norm)	FP32 累加	避免小值累加误差
Loss 函数	FP32	需要大动态范围

表 8: 不同操作类型的推荐数值精度

11.5 本章小结

高性能 GPU 编程不仅是关于写内核，更是关于**正确的开发方法论**：Profile 驱动的优化、理解异步执行、选择合适的维度和精度。这些工程实践建议在日常开发中往往比手写内核更能带来性能收益。

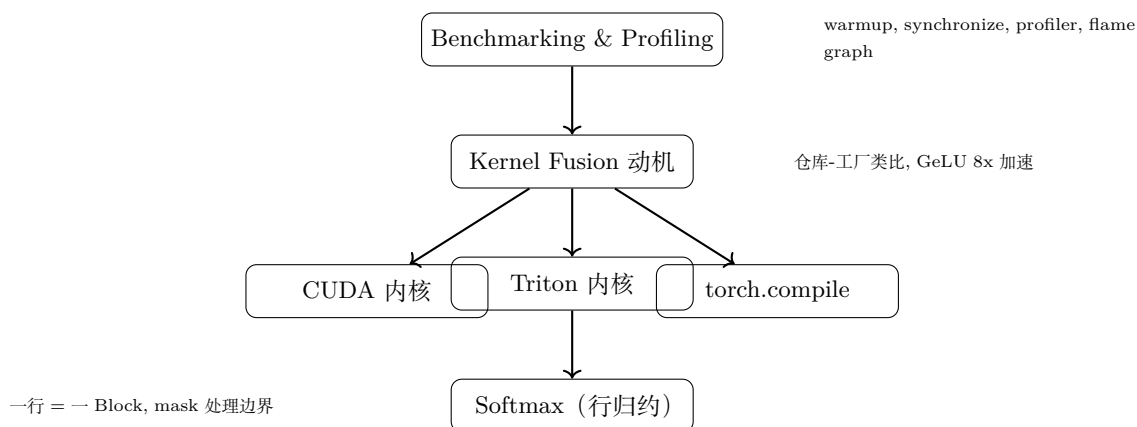
12 总结与延伸

12.1 五种实现方式的权衡

	Manual	PyTorch	torch.compile	CUDA	Triton
开发难度	最低	低	低	高	中
性能	最差	好	很好	好-很好	好-很好
灵活性	高	受限	中等	最高	高
可调试性	好	一般	差	差	好
自动融合	否	已融合	是	否	否

表 9: 五种实现方式的多维度对比

12.2 全课知识图谱



12.3 关键 Takeaways

六条核心原则

1. **编程模型与硬件之间有巨大鸿沟**: 性能之谜源于此, 理解这一点是性能优化的前提
2. **始终基准测试和 Profile**: 不要凭直觉判断瓶颈, 让数据说话
3. **核心原则是最小化读写**: 内核融合、tiling、重计算本质上都是在减少全局内存访问
4. **torch.compile 是低垂的果实**: 对标准操作, 优先使用编译器自动优化
5. **Triton 是自定义内核的首选**: Python 开发体验 + 接近 CUDA 的性能
6. **自动编译器会越来越好**: 今天需要手写的优化, 明天可能由编译器自动完成

12.4 拓展阅读

- Horace He 博客: https://horace.io/brrr_intro.html ——GPU 性能优化入门必读
- Triton 官方教程 (Fused Softmax): <https://triton-lang.org/main/getting-started/tutorials/02-fused-softmax.html>

- Triton 官方教程(Matrix Multiplication): <https://triton-lang.org/main/getting-started/tutorials/03-matrix-multiplication.html>
- Triton 论文: Tillet et al., <https://www.eecs.harvard.edu/~htk/publication/2019-mapl-tillet-kung-co.pdf>
- CUDA MODE Lecture 1 (如何 Profile CUDA 内核) : <https://www.youtube.com/watch?v=LuhJEEJQgUM>
- CUDA MODE Lecture 2(PPMP 书 ch.1-3): <https://www.youtube.com/watch?v=NQ-0D5Ti2dc>
- CUDA MODE Lecture 3 (CUDA for Python Programmers) : <https://www.youtube.com/watch?v=4sgKnKbR-WE>
- CUDA MODE Lecture 4(Compute and Memory Basics): <https://www.youtube.com/watch?v=1TmYrKwjSOU>
- CUDA MODE Lecture 8(CUDA Performance Checklist): <https://www.youtube.com/watch?v=SGhfUhlowB4>
- HetSys Course (GPU Programming) : <https://www.youtube.com/watch?v=8JGo2zylE80>
- GPU Puzzles: <https://github.com/srush/gpu-puzzles>
- NVIDIA Deep Learning Performance Guide: <https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html>
- PyTorch 2.0 Operator Fusion:
<https://towardsdatascience.com/how-pytorch-2-0-accelerates-deep-learning-with-operator-fusion-and-cpu-gpu>